get started, create a file located at lib/phone_app/conversations/query/contact_store.ex and add the following code:

phone_app/lib/phone_app/conversations/query/contact_store.ex
```
defmodule PhoneApp.Conversations.Query.ContactStore do
  alias PhoneApp.Repo
  alias PhoneApp.Conversations.Schema.Contact
end
```

We'll use these aliases throughout our module. Let's add the first function to fetch a contact by its id:

phone_app/lib/phone_app/conversations/query/contact_store.ex
```
def get_contact!(id) do
  Repo.get!(Contact, id)
end
```

This function uses the PhoneApp.Repo.get!/2 function to retrieve an instance of the specified schema module by its primary key. If a record can't be found, an error will be raised. Try it out:

```
$ iex -S mix
iex> iex(9)> PhoneApp.Conversations.Query.ContactStore.get_contact!(1)
** (Ecto.NoResultsError) expected at least one result but got none in query:

from c0 in PhoneApp.Conversations.Schema.Contact,
  where: c0.id == ^1
```

If we were to use PhoneApp.Repo.get/2 instead, we would receive a nil response. The ! in the function name communicates that the function will raise an error. This pattern is idiomatic in Ruby and ActiveRecord, so it should feel familiar.

The next code is a bit more complex. Type it out, and then we'll walk through it:

phone_app/lib/phone_app/conversations/query/contact_store.ex
```
def upsert_contact(%{from: from, to: to, direction: direction}) do
  contact_number =
    case direction do
      :incoming -> from
      :outgoing -> to
    end

  cs = Contact.changeset(%{phone_number: contact_number})

  Repo.insert(
    cs,
    returning: true,
    on_conflict: {:replace, [:updated_at]},
    conflict_target: [:phone_number]
  )
end
```